CloudVault — Real Loopholes & Faculty Q&A Guide

PID: 23CSEP03 — PPD-II Third Review — November 2026

Important

There **are** real security loopholes in CloudVault. Being upfront about them shows technical maturity. Below is the honest analysis + how to answer each question confidently.

The 3 REAL Loopholes

Loophole 1: Server Sees Plaintext During Upload

The user uploads a **plaintext file** to the server, and the **server encrypts it**. This means the server momentarily has access to the original file. A truly end-to-end encrypted system would encrypt the file **in the browser before upload**.

Impact: A compromised or malicious server could save/read files before encrypting them.

What to say: *"We chose server-side encryption for simplicity. In a production system, we would add client-side encryption before upload using the Web Crypto API — the same API we already use for decryption."*

Loophole 2: Server Holds a Copy of Every AES Key (SERVER Entry)

The Key Envelope Table stores a `SERVER` entry for each file — the AES key encrypted with the server's RSA public key. This means the **server can decrypt any file at any time** using its own RSA private key.

Impact: This is NOT truly zero-knowledge. The server has the ability to access all files.

What to say: *"The SERVER key entry exists to enable sharing without involving the file owner. This is a deliberate trade-off: we chose usability (zero-effort sharing) over pure zero-knowledge. In a stricter model, the owner would need to be online to re-wrap the AES key for each recipient."*

Loophole 3: Losing Browser Data = Losing Private Key Forever

The RSA private key is stored **only** in IndexedDB. If the user clears browser data, uses a different browser, or formats their PC — **all their files become permanently inaccessible**.

Impact: No key recovery or backup mechanism exists.

What to say: *"This is a known limitation. Our next step would be implementing encrypted key export — the user downloads their private key as an encrypted file (protected by a passphrase) that they can import on another device."*

Answers to All 10 Faculty Questions

Q1: What are the potential loopholes in your project?

Answer: There are three main ones:

1. The server sees plaintext files during upload (not true E2E encryption)

2. The server holds a copy of the AES key for each file (SERVER entry), so it *can* decrypt files

3. If the user clears browser data, their RSA private key is permanently lost (no export/backup feature)

These are **deliberate design trade-offs** — server-side encryption simplifies the architecture, the SERVER key enables zero-effort sharing, and browser-only keys ensure the private key never travels over the network.

Q2: What are the major security vulnerabilities in CloudVault?

Answer:

- **XSS (Cross-Site Scripting):** If an attacker injects malicious JavaScript into the page, they could access IndexedDB and steal the RSA private key. *Mitigation: React auto-escapes output, and we don't use* `dangerouslySetInnerHTML`.
- **No rate limiting:** The login endpoint has no brute-force protection. *Mitigation: BCrypt's intentional slowness makes brute-force impractical, but adding rate limiting would be better.*
- **JWT in localStorage:** Vulnerable to XSS. *Mitigation: Using httpOnly cookies would be more secure, but complicates CORS.*
- **H2 database:** Not suitable for production. The database file on disk is unencrypted.

Q3: Can an attacker bypass your encryption mechanism?

Answer: No, the encryption itself (AES-256-GCM + RSA-2048-OAEP) is **cryptographically secure** and cannot be brute-forced with current technology. However, an attacker doesn't need to break the encryption if they can:

- Compromise the **server** (which holds SERVER key entries)
- Exploit an **XSS vulnerability** to steal the private key from IndexedDB
- Intercept the **plaintext file during upload** (before server encrypts it)

The encryption algorithm is strong; the potential weaknesses are in the **architecture around it**.

Q4: What happens if the user's browser gets compromised?

Answer: This is serious. If an attacker has access to the user's browser (via XSS or malware):

- They can read the **RSA private key from IndexedDB**
- They can intercept the **JWT token from localStorage**
- With both, they can download and decrypt **all of the user's files**

Mitigation we'd add:

- Content Security Policy (CSP) headers to prevent XSS
- Subresource Integrity (SRI) for all scripts
- Encrypting the IndexedDB private key with a passphrase

Q5: Can an attacker steal the AES encryption key?

Answer: The AES key is **never stored in plaintext** — it only exists in encrypted (wrapped) form in the database. To steal it, an attacker would need:

- The **user's RSA private key** (to unwrap the OWNER entry) — stored only in the browser
- OR the **server's RSA private key** (to unwrap the SERVER entry) — stored as a file on the server

If neither key is compromised, the AES key cannot be recovered.

Q6: What happens if the RSA private key is compromised?

Answer:

- **User's private key compromised:** Attacker can decrypt all files owned by or shared with that user. *No forward secrecy — all past files are exposed.*
- **Server's private key compromised:** Attacker can decrypt the SERVER key entry for **every file in the system** → full breach.

Mitigation: Key rotation. Periodically generate new RSA keys and re-encrypt all key entries. We haven't implemented this yet, but it's the standard solution.

Q7: Can your system prevent all types of cyberattacks?

Answer: No system can prevent all cyberattacks. CloudVault protects against:

✓ Unauthorized file access (JWT + per-user key entries)

✓ Data at rest exposure (AES-256-GCM encryption)

✓ Password theft (BCrypt hashing)

✓ Man-in-the-middle (HTTPS + encrypted file transfer)

It does **NOT** protect against:

× Browser-level attacks (XSS, malware)

× Server compromise (server holds key copies)

× Social engineering

× DDoS attacks (no rate limiting)

Q8: What are the weaknesses of your current security architecture?

Answer:

1. **Not true E2E encryption** — server encrypts, so it sees plaintext during upload
2. **Server is a single point of trust** — it holds AES key copies and can decrypt any file
3. **No key backup/recovery** — browser data loss = permanent file loss
4. **No key rotation** — compromised keys expose all historical files
5. **No audit logging** — no record of who accessed what and when
6. **H2 database** — unencrypted, single-file, not production-grade

Q9: Can an attacker access files after access revocation?

Answer: After revocation, the SHARED key entry is **deleted from the database**, so the recipient can no longer request the encrypted AES key from the server. However:

- If the recipient **already downloaded and decrypted** the file before revocation, they have the plaintext — revocation cannot undo that.
- If the recipient **cached the decrypted AES key** in memory during their session, they could still decrypt the file.
- The **file itself is NOT re-encrypted** after revocation (only the key entry is deleted).

What to say: *"Revocation is effective for future access but cannot retroactively protect data that was already downloaded. This is a fundamental limitation shared by all file-sharing systems, including Google Drive and Dropbox."*

Q10: What are the possible loopholes in your file-sharing mechanism?

Answer:

1. **Server mediates sharing** — the server decrypts the AES key to re-wrap it. A truly zero-knowledge system would use proxy re-encryption so the server never sees the plaintext AES key.
2. **No sharing permissions** — shared users get full read access; there's no "view-only" or "time-limited" sharing.
3. **No notification** — the recipient isn't notified when a file is shared with them.
4. **Trust in server** — the server could share files with any user without the owner's knowledge (since it holds the SERVER key entry).

What To Do Next — Improvements

If faculty asks "what would you improve?", mention these:

Priority	Improvement	Description
High	Client-side encryption before upload	True E2E — encrypt in browser before sending to server
High	Encrypted private key export/import	Backup key as passphrase-protected file
Medium	Rate limiting on login	Prevent brute-force attacks
Medium	Move JWT to httpOnly cookies	Prevent XSS token theft
Medium	Audit logging	Record who accessed what and when
Low	Key rotation mechanism	Re-encrypt keys periodically
Low	Proxy re-encryption for sharing	Eliminate SERVER key entry entirely
Low	Switch H2 to PostgreSQL	Production-ready database

Tip

Faculty impression strategy: Don't hide the loopholes. Say: *"We identified these limitations during development and made conscious trade-offs between security and usability. Here's what we'd improve in the next iteration..."* This shows you **understand** security, not just code.